PredictX · TEAM PITCH DECK

# Jack Schneider

PROJECT

**Smart Energy Guardian**

THEME

Energy Intelligence & Sustainability

*AI-based IoT system that finds and stops electricity wasted in empty campus rooms*

**Software prototype: working**

**Hardware build: in progress**

BRANCH

Electrical & Electronics Engineering (EEE)

TEAM SIZE

*3 Members — Suyash · Arjun · Arya*

FACULTY MENTOR

Ravichandra Halagatti

---

*Powering Predictive Intelligence — A PredictX Campus Hackathon Submission*

# Smart Energy Guardian

*What real-world problem are you solving, and for whom?*

Institutional buildings (classrooms, labs, hostels) waste huge amounts of electricity because lights, fans and ACs keep running in empty rooms. Nobody notices until the bill arrives — facility teams have no room-level visibility, causing financial loss, extra carbon emissions and structural inefficiency.

## OUR SOLUTION

A low-cost IoT + AI system that detects, analyzes and eliminates electrical energy wastage in institutional campuses.

An ESP32 in each room collects real-time data from PIR/mmWave occupancy sensors, SCT-013/PZEM current & power sensors and temperature modules every 5 seconds.

If a room is vacant but appliances keep drawing power, the system passes a set of safety checks, executes an automated relay shut-off and sends real-time alerts to a centralized React.js web dashboard.

**Output: live power, kWh used and wasted, cumulative kWh savings, ₹ cost reduction and estimated $CO_2$ offset — per room and campus-wide.**

## INTELLIGENCE ENGINE

Edge-AI Intelligence Engine: an Isolation Forest anomaly detection model flags abnormal electrical behaviour — spikes, over-consumption, voltage faults, low power factor and phantom loads.

A deterministic decision engine combines occupancy, vacancy time, class schedules, manual overrides and appliance rules, so necessary loads are separated from outright waste — no false positives or blind shut-offs like binary motion sensors.

Hybrid edge-cloud: the ESP32 acts locally (relay, fail-safe), the server runs analytics, and the dashboard updates live over WebSocket.

## TECH STACK

- ESP32 microcontroller
- PIR / mmWave occupancy
- SCT-013 / PZEM-004T power sensing
- Temperature modules
- Relay-based shut-off
- Isolation Forest ML (scikit-learn)
- Python FastAPI backend
- SQLite / PostgreSQL
- React.js + TypeScript dashboard

# Why It Matters

PROBLEM & IMPACT

*The cost of one forgotten classroom — and of a whole campus*

### ONE TYPICAL CLASSROOM

- 6 tube lights × 40 W = 240 W
- 4 ceiling fans × 75 W = 300 W
- 1 split AC ≈ 1,500 W
- Total ≈ 2.04 kW left running
- Empty but ON ~3 h per day
- 220 working days per year
- Tariff ₹8/kWh · grid 0.71 kg $CO_2$/kWh

*Illustrative estimate based on these assumptions — the exact figure depends on each campus.*

**1,346 kWh**

wasted per room / year

**₹10,771**

lost per room / year

**956 kg**

$CO_2$ per room / year

### SCALE IT TO 100 ROOMS

**≈ 1.35 lakh kWh · ≈ ₹10.8 lakh · ≈ 96 tonnes $CO_2$**

every single year — from rooms where nobody is even present. In our 7-day simulated demo campus, 15.9% of all energy was drawn in vacant rooms.

# Innovation & Differentiation

INNOVATION & ORIGINALITY · 25%

*What makes your approach genuinely new?*

## TODAY'S APPROACH

- Appliances are switched off manually by occupants or staff, or run on fixed timers — so empty rooms keep drawing power.
- Conventional binary motion sensors trigger false positives or blind shut-offs (e.g. a lab cut off mid-experiment) and cannot tell necessary loads from waste.
- No room-level visibility: savings are never measured, and faults like a stuck relay or phantom load go unnoticed.
- Commercial building-management systems are costly and built for large offices, not classrooms and hostels.

## YOUR APPROACH

1. **Edge-AI Intelligence Engine:** ML anomaly detection on electrical features catches spikes, over-consumption, voltage faults and phantom loads.
2. **Multi-signal sensing:** occupancy (PIR/mmWave), current & power (SCT-013/PZEM) and temperature fused on a low-cost ESP32.
3. **Explainable, safety-first decisions:** 20 checks (data freshness, schedules, overrides, essential loads) — every decision is shown check-by-check.
4. **Closed loop:** relay shut-off with acknowledgement & timeout, real-time alerts and a dashboard tracking kWh, ₹ and $CO_2$ savings.
5. **Hardware-ready, deploy-anywhere:** a simulator and ESP32 emulator use the exact device protocol, so the software is tested before a single wire is connected.

# How It Works — System Architecture

TECHNICAL DESIGN

*Four layers, one data path from sensor to screen*

### 1. Room Node (Edge)

- ESP32 + PIR + PZEM/SCT-013 + relay board
- Sends readings every 5 s over Wi-Fi (HTTPS JSON)
- Switches relay ON/OFF and confirms with the power meter

### 2. Backend (Python · FastAPI)

- Telemetry pipeline: kWh, waste, cost, $CO_2$
- ML anomaly scoring of every reading
- Decision engine + command service
- Monitor loop every 3 s (offline, timeouts)

### 3. Database

- SQLite (local) or PostgreSQL / Supabase (cloud)
- Rooms, appliances, readings, 15-min energy buckets, alerts, commands, audit log

### 4. Dashboard (React · TS)

- Live updates over WebSocket
- 12 pages: rooms, alerts, history, ML, simulation, settings
- Secure login: viewer / operator / admin

**Tested today without hardware:** a built-in simulator (5 virtual rooms, 7 scenarios) and an ESP32 emulator talk to the backend using the exact same protocol as the real ESP32 — so when our hardware is ready, the software needs no changes.

# Safety-First Decision Engine

TECHNICAL DESIGN

*An appliance is switched off only when every check passes*

## 1 Data Quality

**Is the data fresh and trustworthy?**

Power reading < 45 s old, occupancy known, sensor OK, ESP32 online.

*If not → do nothing*

## 2 Wastage

**Is energy really being wasted?**

Appliance above standby power AND room vacant for the minimum time (10 min).

*If not → no action*

## 3 Safety Policy

**Is it safe to switch off?**

Not an essential load, no manual override, no class scheduled now or in 10 min, no recent manual switch-on.

*If not → only recommend*

## 4 Mode

**What are we allowed to do?**

Recommend → alert only. Dry-run (default) → log only. Auto → send OFF command.

*Relay must acknowledge in 20 s*

**Real relay switching needs 3 keys:** Auto mode + hardware switching enabled on the server + that ESP32 validated by an admin.

**AI advises, rules decide:** an ML anomaly never switches anything off on its own — it raises an alert for a human to inspect.

# ML Anomaly Detection

AI / ML

*Isolation Forest — learns what normal electrical behaviour looks like and flags what doesn't*

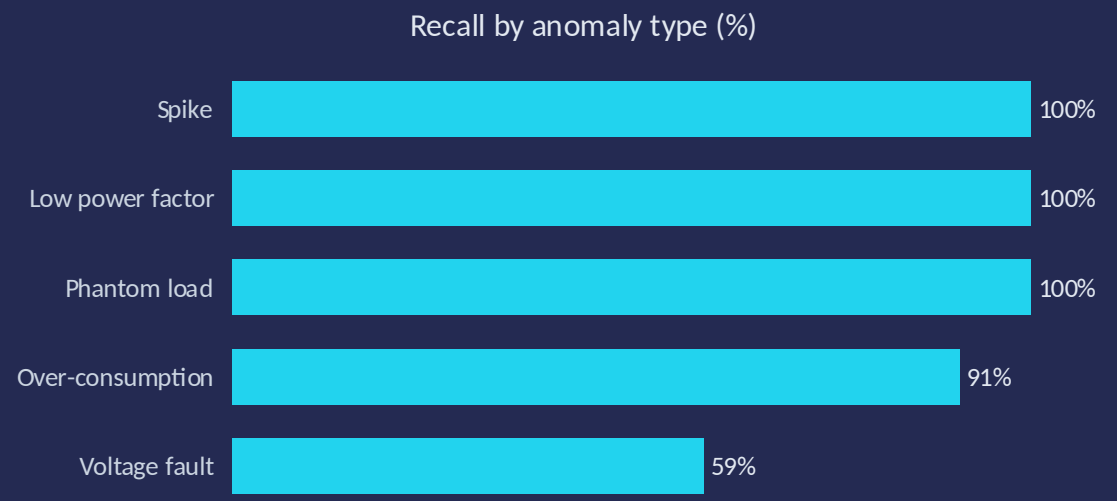
## WHAT IT CATCHES

- Over-consumption (load far above its rating)
- Sudden power spikes
- Voltage faults
- Low power factor (inefficient load)
- Phantom load — power flowing while the relay says OFF

## HOW IT WORKS

- 5 input features: power ratio, step change, voltage deviation, power factor, relay mismatch
- 200 trees, trained on 16,000 readings, tested on 8,000 unseen readings
- Alert only after 2 flagged readings in a row (no one-off noise)
- Rule-based fallback if no model is loaded

| 0.995 | 86.7% | 70.1% | 1.1% |
|---|---|---|---|
| ROC-AUC | Recall | Precision | False alarms |

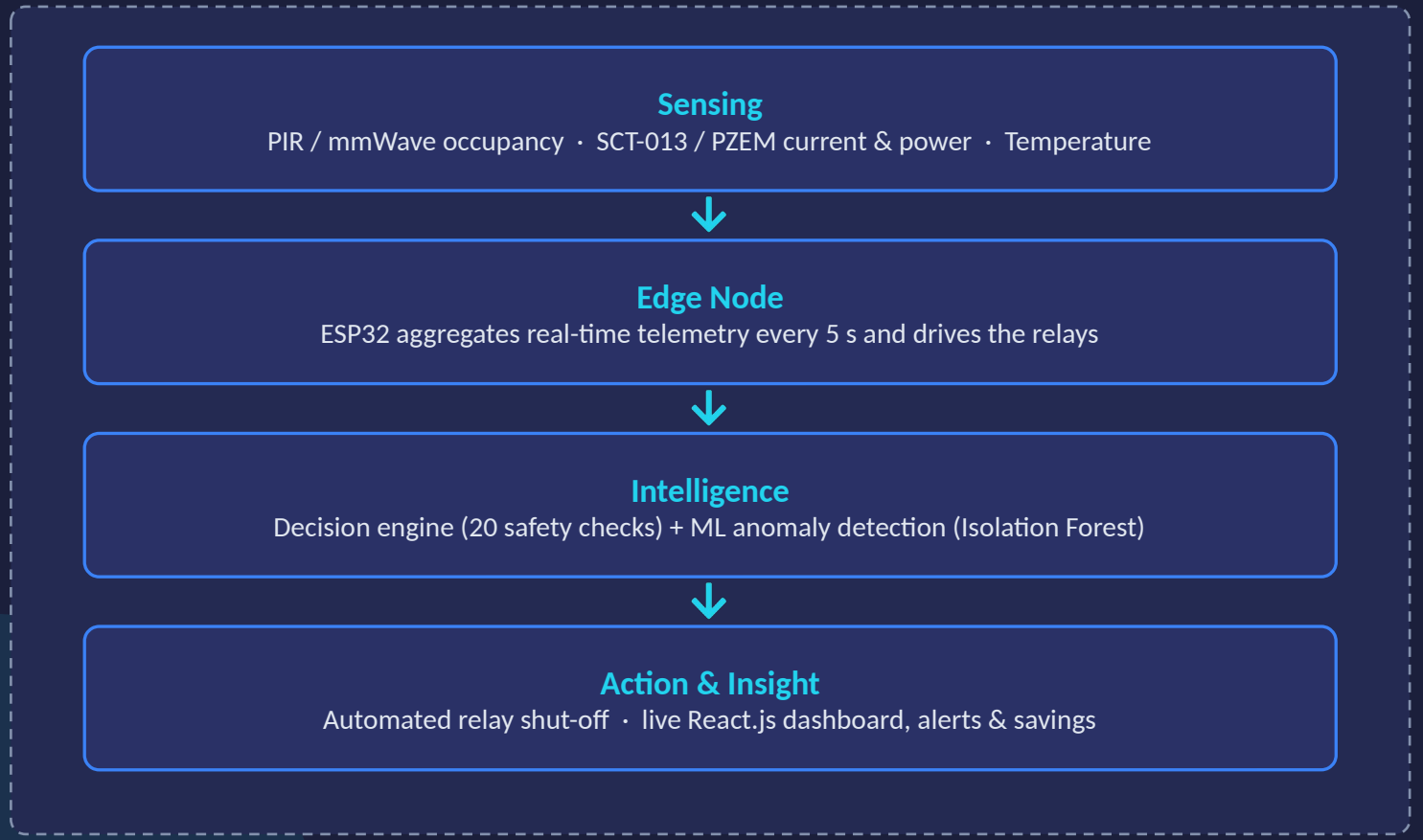



*Measured on synthetic data from our own simulator — it proves the pipeline works. Real accuracy will be measured once hardware data is collected.*

# Prototype & Demo

PRESENTATION & PROTOTYPE · 20%

*Show, don't just tell — this is your proof of feasibility*




## DEMO NOTES

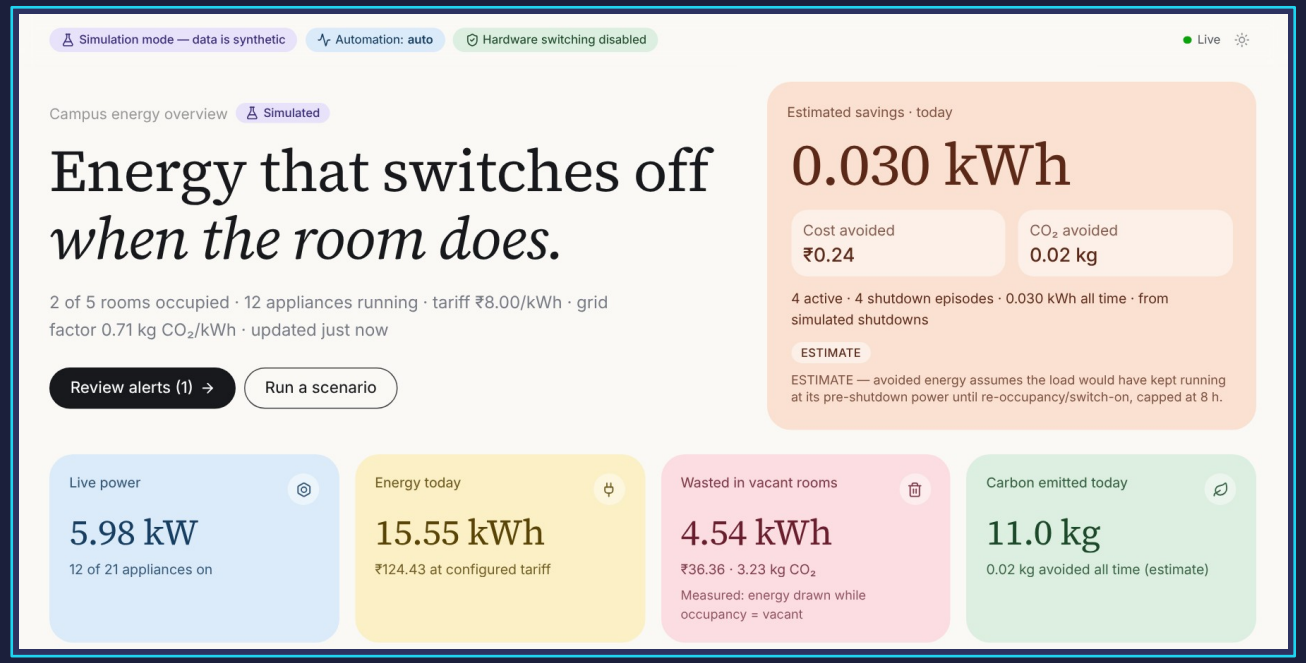
**Stage:** working software demo (simulator + ESP32 emulator). Hardware node: in progress.

- Detects vacant rooms with abnormal power draw, cuts power via relay (simulated) and alerts the dashboard in real time.
- Demo scenarios: vacant-room waste, anomaly, sensor failure, offline device, failed and timed-out commands.
- 127 automated tests passing (105 backend, 18 frontend, 4 end-to-end).
- Next: wider rollout across labs and hostels, deeper per-room analytics and mobile alerts.

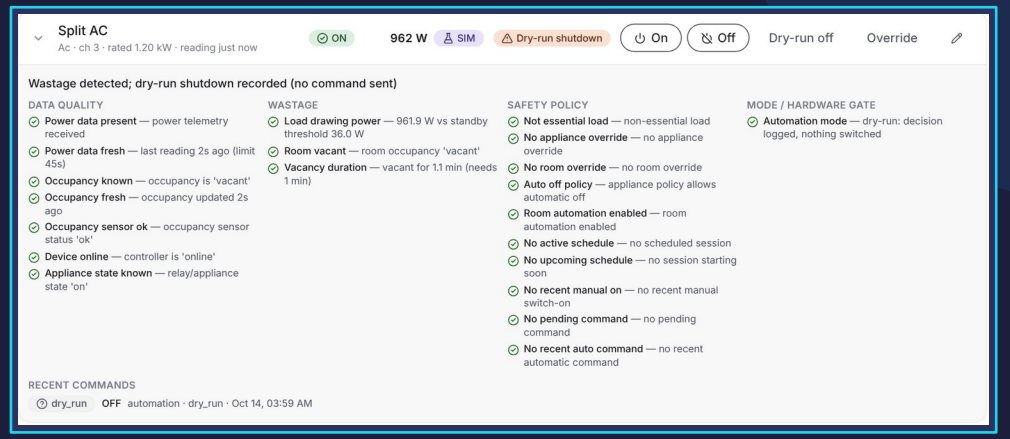
# Software Prototype — Live Dashboard

WORKING SOFTWARE

*Real screens from our running software (simulated campus data)*




*Campus overview: live power, energy today, energy wasted in vacant rooms, $CO_2$ and estimated savings*




*Room view: every decision explained check-by-check (here: Split AC in an empty seminar room)*

**Labelled honestly:** every screen shows whether data is Simulated, Emulated ESP32 or Hardware, and savings are always marked as estimates.

## WHAT THE DASHBOARD GIVES

- Live kW, kWh, ₹ cost and $CO_2$ — per room and campus
- Energy wasted while rooms were empty
- Alerts with recommendations · manual ON/OFF & overrides
- Schedules, devices, users & roles, settings

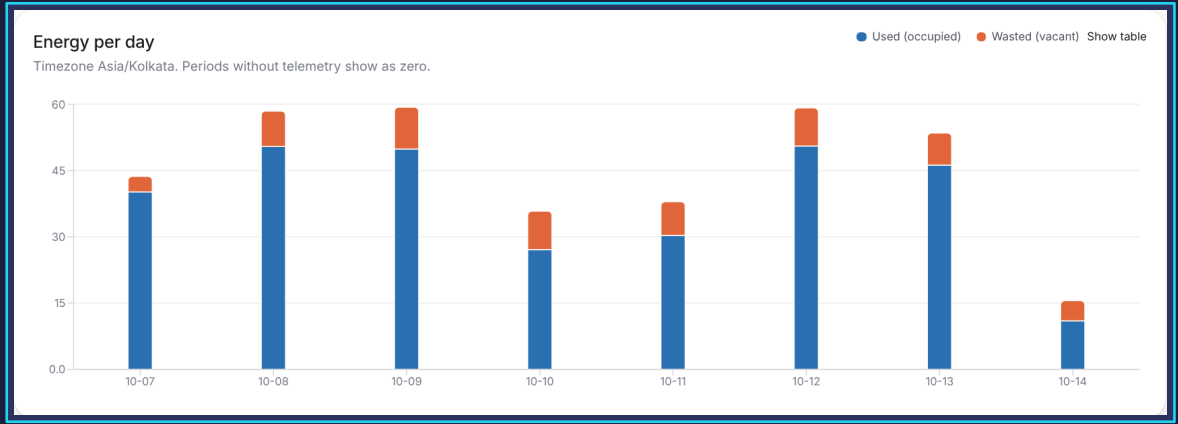
# Software Prototype — Automation & History

WORKING SOFTWARE

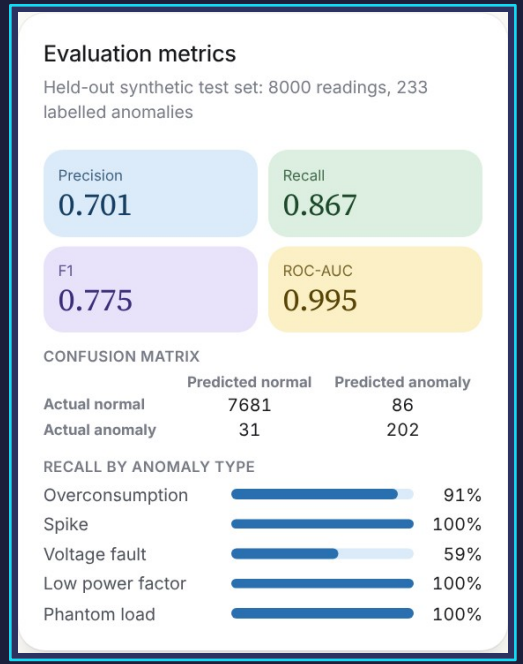
*From detection to shutdown to savings — all tracked*

| # | Appliance | Action | Status | Mode | Origin | Created | Sent | Completed | Reason / error |
|---|---|---|---|---|---|---|---|---|---|
| 8 | Split AC | OFF | acknowledged | simulated | automation | 04:00:09 AM | 04:00:12 AM | 04:00:15 AM | Wastage detected; simulated shutdown command issued |
| 7 | Projector | OFF | acknowledged | simulated | automation | 04:00:09 AM | 04:00:12 AM | 04:00:15 AM | Wastage detected; simulated shutdown command issued |
| 6 | Ceiling fans | OFF | acknowledged | simulated | automation | 04:00:09 AM | 04:00:12 AM | 04:00:15 AM | Wastage detected; simulated shutdown command issued |
| 5 | Lights bank | OFF | acknowledged | simulated | automation | 04:00:09 AM | 04:00:12 AM | 04:00:15 AM | Wastage detected; simulated shutdown command issued |
| 4 | Split AC | OFF | dry-run | dry-run | automation | 03:59:59 AM | — | 03:59:59 AM | Wastage detected; dry-run shutdown recorded (no command sent) |
| 3 | Projector | OFF | dry-run | dry-run | automation | 03:59:59 AM | — | 03:59:59 AM | Wastage detected; dry-run shutdown recorded (no command sent) |
| 2 | Ceiling fans | OFF | dry-run | dry-run | automation | 03:59:59 AM | — | 03:59:59 AM | Wastage detected; dry-run shutdown recorded (no command sent) |
| 1 | Lights bank | OFF | dry-run | dry-run | automation | 03:59:59 AM | — | 03:59:59 AM | Wastage detected; dry-run shutdown recorded (no command sent) |

*Command log: dry-run decisions, then automatic OFF commands acknowledged by the controller*




*Energy history: used (blue) vs wasted in vacant rooms (orange), 7 days*




*ML page: metrics, confusion matrix, recall by type*

## DEMO IN 4 STEPS

1. Room left ON while empty → waste alert appears
2. Dry-run shows what would be switched off
3. Switch to Auto → OFF commands sent & acknowledged
4. Savings counter starts; history shows waste dropping

# Project Status — Where We Stand

PROGRESS

*Software complete for this review · hardware build planned for the final review*

## SOFTWARE COMPLETED

- Backend: data pipeline, energy & cost maths, alerts, commands
- Safety-first decision engine with explanations
- ML anomaly model trained, evaluated, versioned
- React dashboard — 12 pages, live updates, dark mode
- Simulator (7 scenarios) + ESP32 emulator
- Docker, cloud deployment setup, CI pipeline

| 57 | 12 | 127 |
|---|---|---|
| REST APIs | UI pages | tests passing |

## HARDWARE IN PROGRESS

### READY

- Components selected & bill of materials
- Wiring / pin plan for the ESP32 node
- ESP32 firmware code drafted
- Device protocol tested end-to-end via emulator

### NEXT — FOR FINAL REVIEW

- Procure parts, assemble and bench-test one room node
- Connect to dashboard in dry-run, then live relay demo
- Collect real readings to re-check the ML model

# Hardware Plan — Toward the Final Review

HARDWARE ROADMAP

*One low-cost room node, designed to fail safe*

| Component | Role in the node |
|---|---|
| **ESP32 DevKit (WROOM-32)** | Brain of the node + Wi-Fi |
| **PIR HC-SR501 / mmWave** | Occupancy — is anyone in the room? |
| **PZEM-004T + CT clamp / SCT-013** | Voltage, current and power per appliance |
| **4-channel opto-isolated relay** | Switches loads (normally-closed = fail-safe) |
| **HLK-PM01 5 V supply** | Powers the node from mains |
| **DHT22 / DS18B20** | Room temperature (add-on for AC logic) |

**Safety by design:** if the node crashes or loses power, the relay's normally-closed contact keeps lights and fans ON; OFF is confirmed with the power meter; mains wiring done under lab supervision.

BUILD PLAN

1. **Procure**
   Buy ESP32, sensors, relay board and power supply
2. **Assemble & flash**
   Wire one node; flash firmware; bench-test boot, Wi-Fi loss, relay check
3. **Connect**
   Register the ESP32 on the dashboard; run in dry-run mode
4. **Live demo**
   Real room: vacant → alert → relay OFF → savings, at the Final Review

# Meet the Team

*Cross-disciplinary skills behind the idea*



### Suyash Tatiya

*24BEE0087*

**Team Lead**

Backend, decision engine & ML model

### Arjun Gupta

*24BEE0064*

**Software & Testing**

React dashboard, simulation, testing & deployment

### Arya Suman

*24BEE0032*

**Research & Hardware**

Problem research, energy analysis & hardware design

*Faculty Mentor:* ***Ravichandra Halagatti***  *Schneider Mentor: assigned during mentoring phase*

# Who Did What

TEAM CONTRIBUTIONS

*Specific tasks owned by each team member*



## Suyash Tatiya

Team Lead · Backend & ML

- System architecture & database design
- FastAPI backend and 57 REST APIs
- Safety-first decision engine (20 checks)
- Isolation Forest pipeline: data, training, metrics
- Command flow: acknowledgement & timeouts

**Presents Part 2: How it works**



## Arjun Gupta

Frontend, Testing & Deployment

- React + TypeScript dashboard (12 pages)
- Live WebSocket updates & charts
- Simulation scenarios & ESP32 emulator runs
- Unit + end-to-end (Playwright) tests
- Docker & cloud deployment (Vercel, Render)

**Presents Part 3: Software & status**



## Arya Suman

Research, Energy & Hardware

- Problem research & campus energy study
- Energy, cost & $CO_2$ calculation model
- Sensor selection & bill of materials
- ESP32 wiring plan & firmware protocol
- Leads hardware build for Final Review

**Presents Part 1: The pitch**

# Roadmap & Next Steps

*Where this goes beyond the hackathon*



**1** **If Shortlisted**

Build and bench-test the ESP32 room node, calibrate the anomaly model on real room-level consumption data, harden the relay and alert logic, and polish the dashboard for the final pitch.

**2** **If Selected as Winner**

Use the Schneider internship to pilot Smart Energy Guardian in live campus buildings and align it with professional energy-management practice.

**3** **Long-Term Vision**

Scale into a deployable blueprint for smart campuses, hostels and commercial buildings — with mobile alerts, people counting and multi-building reports — to achieve measurable net-zero goals.



# Thank You

*Jack Schneider · Questions Welcome*

Suyash Tatiya · Arju Gupta · Arya Suman | Faculty Mentor: Ravichandra Halagatti

*suyash.tatiya2024@vitstudent.ac.in*